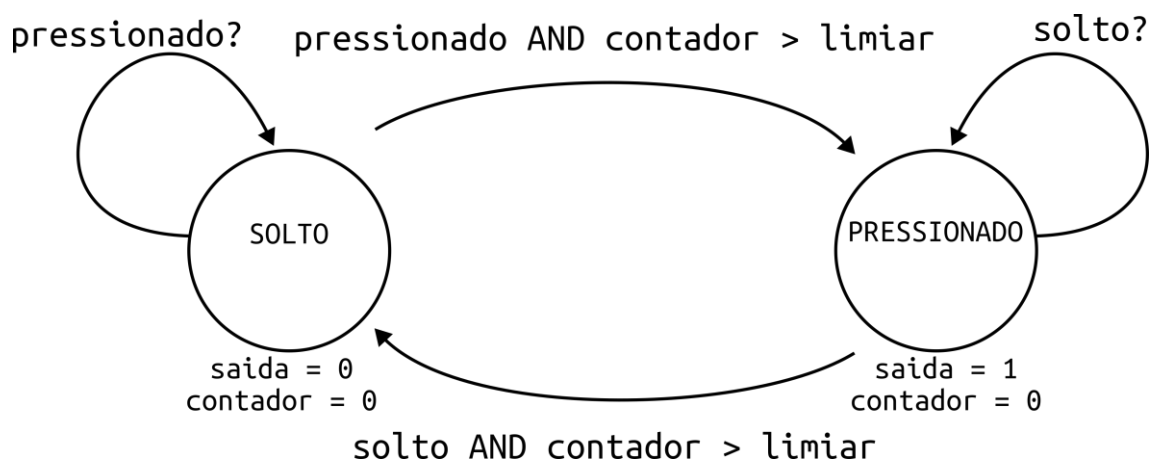


Laboratório 11

Usando uma máquina de estados finita, nós podemos criar um *debouncer* que "limpa" o sinal com ruído dos botões da placa. A ideia é ter apenas dois estados: **solto** ou **pressionado**. Quando o sistema está no estado "solto" e o botão está solto, continuamos no mesmo estado. Quando pressionamos o botão, o circuito incrementa um contador e continua no estado "solto" até que esse contador passe de um limiar, transicionando para o estado "pressionado". Quando estamos no estado "pressionado" e continuamos pressionando o botão, continuamos no mesmo estado. Quando soltamos o botão, o circuito incrementa o contador e continua no estado "pressionado" até que o contador passe de um limiar, voltando ao estado "solto". A imagem a seguir ilustra o diagrama de estados:



O contador é zerado apenas quando entramos em um dos dois estados.

Crie uma entidade com uma FSM para corrigir o código do lab 10.

Os sinais de entrada devem ser:

- Clock (in)
- Reset (in)
- Botao (in)
- Botao_limpo (out)

Use o código do slide 70 do capítulo 8 (aula teórica) como referência para a FSM. Use os estados SOLTO e PRESSIONADO.

Faça o processo sensível ao clock, com reset assíncrono e crie um sinal para o contador do tipo inteiro (signal contador : integer range 0 to 999999 := 0).

Usar 999999 como limiar é suficiente.

A lógica para o estado "SOLTO" é: se o botão estiver em 0 (pressionado), confira se o contador já atingiu o limite, que muda o estado para "pressionado" e o contador para 0. Caso contrário incremente o contador.

A lógica para o estado "PRESSIONADO" é parecida com a anterior: se o botão estiver em 1, confira se o contador já atingiu o limite, que muda o estado para "solto" e o contador para 0. Caso contrário incremente o contador.

Entrega: arquivos vhd comprimidos em um zip. Me chame para mostrar a placa funcionando até o fim da aula (vale metade da nota).